

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot .Net Core and Progressive Web Application Practical

Practical – IX

Roll No.: T002	Name: Asmitha Mohan Raj
Class: TYIT	Batch: 1
Date of Assignment: 18/08/2026	Date/Time of Submission: 18/08/2026

a. Develop CRUD operations using ASP.NET Core Web API and Entity Framework Core.

Code:

Student.cs:

```
namespace StudentAPIWithEF.Models
{
    public class Student
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public int Age { get; set; }
        public string Course { get; set; }
    }
}
```

StudentController.cs:

```
using Microsoft.AspNetCore.Mvc;
using StudentAPIWithEF.Data;
using StudentAPIWithEF.Models;
namespace StudentAPIWithEF.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class StudentsController : ControllerBase
    {
        private readonly AppDbContext _context;
        public StudentsController(AppDbContext context)
        {
            _context = context;
        }

        // GET ALL
        [HttpGet]
        public IActionResult GetStudents()
        {
            return Ok(_context.Students.ToList());
        }

        // GET BY ID
        [HttpGet("{id}")]
        public IActionResult GetStudent(int id)
        {
            var student = _context.Students.Find(id);
            if (student == null)
                return NotFound();
            return Ok(student);
        }
    }
}
```

```
// INSERT
[HttpPost]
public IActionResult AddStudent(Student student)
{
    _context.Students.Add(student);
    _context.SaveChanges();

    return Ok(student);
}

// UPDATE
[HttpPut("{id}")]
public IActionResult UpdateStudent(int id, Student student)
{
    var data = _context.Students.Find(id);
    if (data == null)
        return NotFound();
    data.Name = student.Name;
    data.Age = student.Age;
    data.Course = student.Course;
    _context.SaveChanges();
    return Ok(data);
}

// DELETE
[HttpDelete("{id}")]
public IActionResult DeleteStudent(int id)
{
    var student = _context.Students.Find(id);
    if (student == null)
        return NotFound();
    _context.Students.Remove(student);
    _context.SaveChanges();
    return Ok();
}
}
```

AppDbContext.cs:

```
using Microsoft.EntityFrameworkCore;
using StudentAPIWithEF.Models;
namespace StudentAPI.Data
{
    public class AppDbContext : DbContext
```

```
{
    public
    AppDbContext(DbContextOptions<AppDbContext>
options)
        : base(options)
    {
```

Program.cs:

```
using Microsoft.EntityFrameworkCore;
using StudentAPIWithEF.Data;
public partial class Program
{
    private static void Main(string[] args)
    {
        var builder = WebApplication.CreateBuilder(args);
        builder.Services.AddControllers();

        builder.Services.AddDbContext<AppDbContext>(options
=>
    options.UseSqlServer(

        builder.Configuration.GetConnectionString("DefaultConne
ction")));
        builder.Services.AddEndpointsApiExplorer();
        builder.Services.AddSwaggerGen();
```

```
var app = builder.Build();
app.UseSwagger();
app.UseSwaggerUI();
app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();
app.Run();
    }
}
```

appsettings.json:

```
{
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection":
"Server=(localdb)\MSSQLLocalDB;Database=StudentDB;
Trusted_Connection=True;"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  }
}
```

Output:

Swagger UI interface showing API endpoints for StudentAPIWithEF. The 'Students' section is expanded, showing GET, POST, PUT, and DELETE methods. The PUT endpoint /api/Students/{id} is selected, showing a parameter 'id' with value 2 and a request body containing student details.

SQL Server Enterprise Manager showing the 'dbo.Students' table. The table has columns: Id, Name, Age, and Course. The data row shows Id=2, Name=Asmitha, Age=20, and Course=B.Sc.

b. Create an API endpoint to calculate Factorial of a Number and test it using Postman.

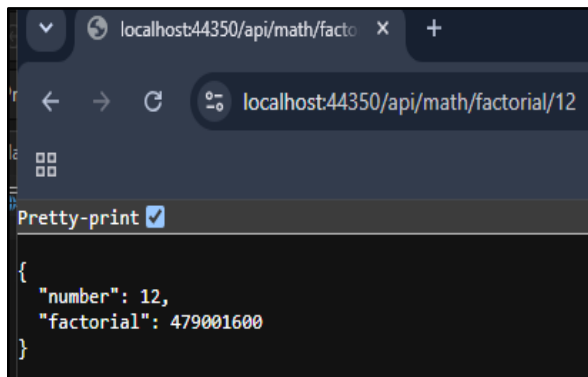
Code:

MathControllers.cs:

```
using Microsoft.AspNetCore.Mvc;
namespace FactorialAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class MathController : ControllerBase
    {
        [HttpGet("factorial/{number}")]
        public IActionResult GetFactorial(int number)
        {
            if (number < 0)
            {
                return BadRequest("Factorial not possible for negative numbers.");
            }
            long factorial = 1;
            for (int i = 1; i <= number; i++)
            {
                factorial *= i;
            }
            return Ok(new
            {
                Number = number,
                Factorial = factorial
            });
        }
    }
}
```

Output:

API in local host:



API using Postman:

